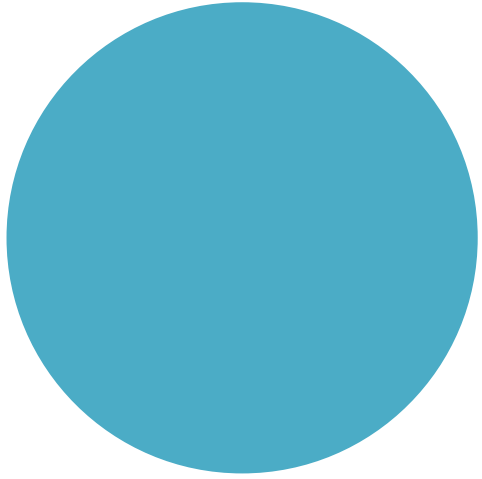


Mobile Devices and Applications

Unit 09 –
Persistent storage

Agenda

Persistent storage

- Application container
- Shared Preferences files
- Internal and External files
- Database - SQLite

Content Providers



APPLICATION CONTAINER

PERSISTENT STORAGE

APPLICATION CONTAINER

Resources

- Directories
 - **res/raw** – any type of files saved in the raw format
 - **res/xml** – XML compiled files
- Uses a resources identifier in order to get access

Assets directory

- Processing data flow
- Used for grouping directories and files

APPLICATION CONTAINER

- **getResources()** -> Resources
 - Gets files inside res/raw
 - **openRawResource()** -> InputStream
 - Gets files from res/xml
 - **getXml()** -> XmlPullParser

APPLICATION CONTAINER

- AssetManager – provides access to an application's raw asset files;
 - **getAssets()** (Context class)
 - **open(*file_name*)** -> InputStream
 - **list(*path*)**
 - Returns a String[] of all assets at the given path
 - Files list
 - **openXmlResourceParser(*file_name*)**
 - Retrieve a parser to an compiled XML file

APPLICATION CONTAINER

- App's private area
 - */data/data/app_package*
- App's private external storage area
 - */ext_dir/Android/data/app_package*
- Subdirectories
 - *cache*
 - *databases*
 - *shared_prefs*
 - *files*



SHARED PREFERENCES

Properties files

Android Preferences

Small Collection of key-value pair

Persistent Data

KEY	VALUE
car_name	Audi S4
color	White
owner	Sriyank

What can and can't do

Store data in XML file as key-value pair

Store only primitive data type

- String
- int
- float
- boolean

Cannot store non-primitive data types

- A custom class object `employee`
- ArrayList
- HashMap

Open mode

File can be Private

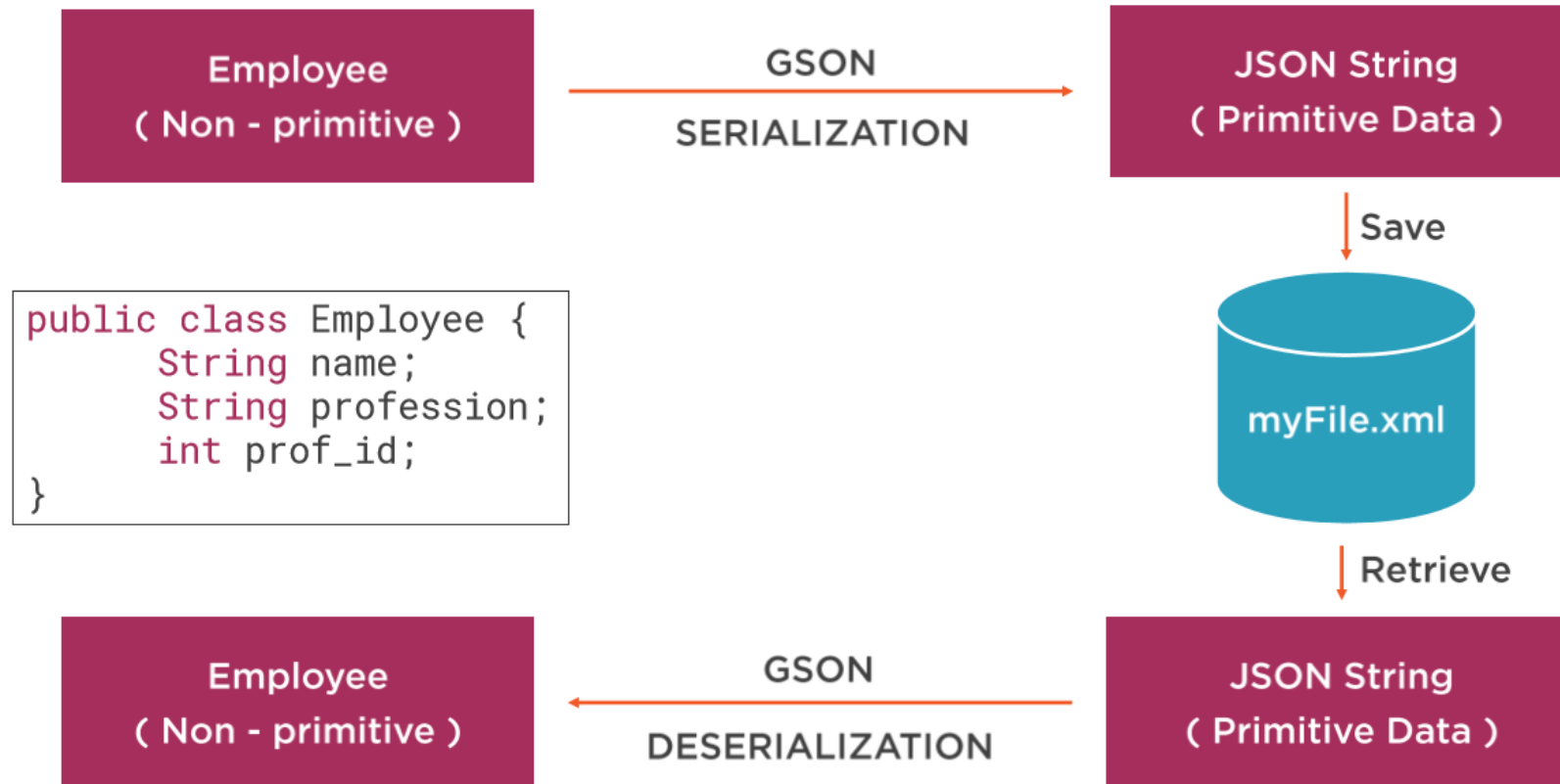
- MODE_PRIVATE

File can be accessible to other apps

- MODE_WORLD_READABLE
 - Deprecated in API 17
- MODE_WORLD_WRITABLE
 - Deprecated in API 17
- Throws `SecurityException` from Android Nougat onwards

Storing non-primitive values

How It Works?



PROPERTIES FILES

- Uses the **SharedPreferences** Interface
- Persistent storage of key-value pair
- Data types allowed
 - boolean
 - int
 - float
 - long
 - String
- *getType()* methods like *getBoolean()*, *getInt()* etc.

PROPERTIES FILES

- At application level
 - **getSharedPreferences**(String file_name, int operating_mode)
 - An application can have multiple preference files
- At activity level
 - **getPreferences**(int mode)
 - A single properties file per application activity
- From PreferenceManager class
 - **getDefaultSharedPreferences**(*context*)
 - The default file for storing preferences in the current context
- Returns an object of type
 - **SharedPreferences**
- Mode
 - Activity.MODE_PRIVATE - default

PROPERTIES FILES

- `SharedPreferences.Editor`
- Initialization
 - `SharedPreferences#edit()`
- Writing
 - `putType()` methods
- Deleting preferences
 - `remove()`
- Delete all preferences
 - `clear()`
- Saving changes
 - `commit()`

PROPERTIES FILES

```
SharedPreferences settings = getSharedPreferences("settings",  
    Activity.MODE_PRIVATE);
```

```
SharedPreferences.Editor editorProp = settings.edit();
```

```
editorProp.putBoolean("title", false);
```

```
editorProp.putBoolean("help", true);
```

```
editorProp.putInt("max", 5);
```

```
editorProp.commit();
```

PROPERTIES FILES

```
SharedPreferences settings = getSharedPreferences("settings",  
        Activity.MODE_PRIVATE);
```

```
boolean fTitle = settings.getBoolean("title", false);
```

```
boolean fHelp = settings.getBoolean("help", true);
```

```
int nMax = settings.getInt("max", 5);
```

Activities for saving preferences

- Classes
 - PreferenceActivity
 - PreferenceFragment
 - PreferenceScreen
 - PreferenceCategory
- Preferences
- Dialog windows
 - EditTextPreference
 - ListPreference
 - MultiSelectListPreference
- Controls
 - CheckBoxPreference
 - SwitchPreference

Activities for saving preferences

- PreferenceActivity
- PreferenceFragment
- XML files associated with content
 - `addPreferencesFromResource(R.xml.preferences);`

Activities for saving preferences

```
<PreferenceScreen  
xmlns:android="http://schemas.android.com/apk/res/android" >
```

```
<PreferenceCategory android:title="Informatii conectare" >
```

```
<EditTextPreference
```

```
    android:key="utilizator"
```

```
    android:summary="Introduceti numele de utilizator"
```

```
    android:title="Utilizator" />
```

```
<EditTextPreference
    android:inputType="textPassword"
    android:key="parola"
    android:negativeButtonText="Renunta"
    android:positiveButtonText="Accepta"
    android:summary="Introduceti parola"
    android:title="Parola" />

<CheckBoxPreference
    android:key="raminConectat"
    android:summary="Se mentine sau nu autentificarea"
    android:title="Ramin conectat" />
</PreferenceCategory>
```

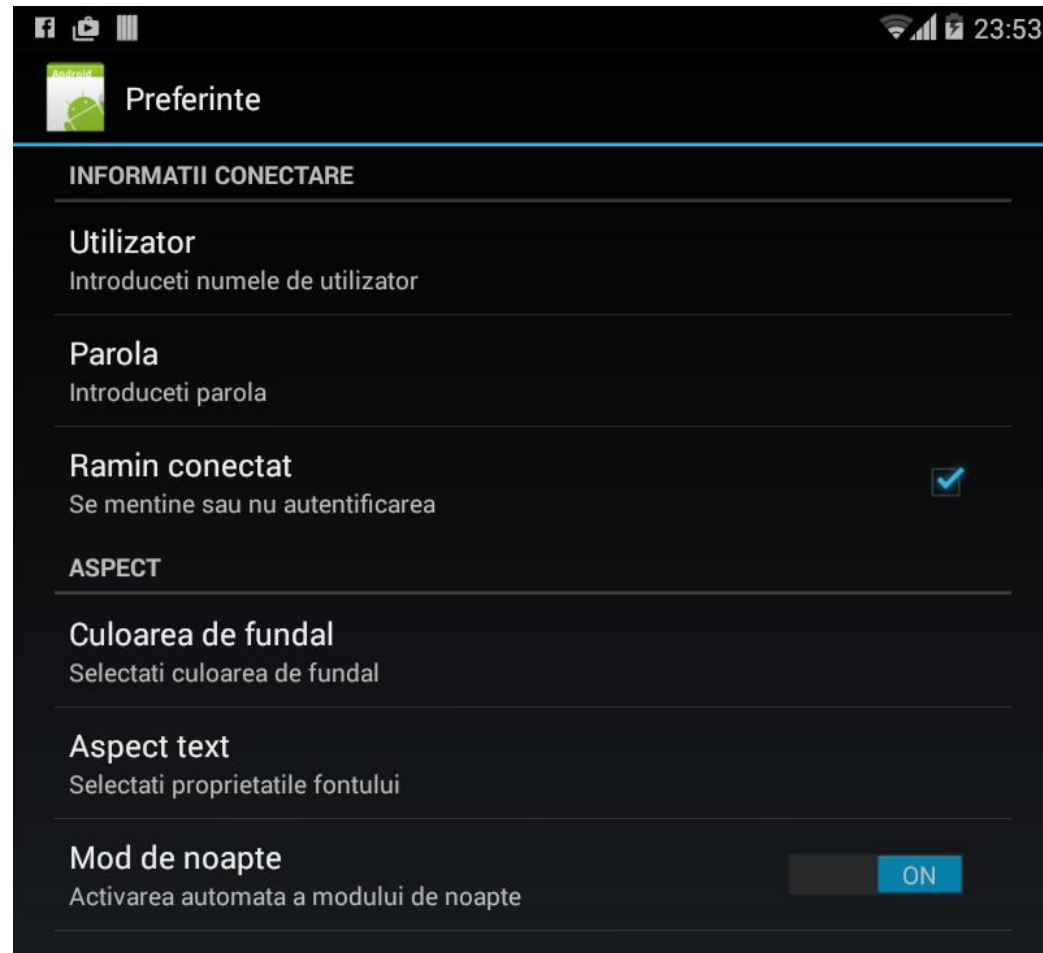
```
<PreferenceCategory
    android:summary="Preferinte cu privire la fonturi si culori"
    android:title="Aspect" >
    <ListPreference
        android:entries="@array/optiuniCulori"
        android:entryValues="@array/culoriDisponibile"
        android:key="listaCulori"
        android:negativeButtonText="Renunta"
        android:summary="Selectati culoarea de fundal"
        android:title="Culoarea de fundal" />
```

```
<MultiSelectListPreference
    android:entries="@array/optiuniFont"
    android:entryValues="@array/setariFontDisponibile"
    android:key="listaFont"
    android:summary="Selectati proprietatile fontului"
    android:title="Aspect text" />

<SwitchPreference
    android:key="modNoapte"
    android:summary="Activarea automata a modului de noapte"
    android:title="Mod de noapte" />
</PreferenceCategory>

</PreferenceScreen>
```

Activities for saving preferences



Activities for saving preferences

```
SharedPreferences preferinte = PreferenceManager  
    .getDefaultSharedPreferences(this);
```

```
boolean modNoapte =  
    preferinte.getBoolean("modNoapte", false);  
String user = preferinte.getString("utilizator",  
    "neconectat");  
String [] setariFonturi = new String[5];  
preferinte.getStringSet("listaFont", new  
    HashSet<String>()).toArray(setariFonturi);
```



FILES

FILES



Can be used as
persistent storage
option

internal
external



The internal files are implicitly
available just from the
application's context



The files are deleted once the
application is uninstalled

Environment class

- External storage access:
 - **isExternalStorageRemovable()**
- External storage state
 - **getExternalStorageState()**
 - MEDIA_MOUNTED
 - MEDIA_MOUNTED_READ_ONLY
 - MEDIA_UNMOUNTED
 - MEDIA_REMOVED

Special directories

- **Environment** class
- Root directory
 - **getRootDirectory()**
- User directory
 - **getDataDirectory()**
- Cache directory
 - **getDownloadCacheDirectory()**
- External directory
 - **getExternalStorageDirectory()**

Special directories

- Public directory for external storage
 - **getExternalStoragePublicDirectory(*type*)**
 - *type* (static members)
 - DIRECTORY_PICTURES
 - DIRECTORY_MUSIC
 - DIRECTORY_DOWNLOADS
 - DIRECTORY_DCIM
 - DIRECTORY_RINGTONES
 - DIRECTORY_ALARMS

Special directories

- Activity class (Context)
- External directory for the application
 - **getExternalFilesDir()**
- External directories for the app (API 19)
 - **getExternalFilesDirs()**
- Data directory
 - **getFilesDir()**
- Internal cache directory
 - **getCacheDir()**
- External cache directory
 - **getExternalCacheDir()**

File class

- File and directory based operations
- Setting properties for each file
- Change or move the objects
- Can create a directory or a file
- Can read the content of a directory or a file

Files

InputStream and OutputStream

- Abstract classes
- Used for data stream operations

FileInputStream and FileOutputStream

- Reading and writing data bytes streams;

FileReader and FileWriter

- Reading and writing char sequences;

Files

- **InputStreamReader** and **OutputStreamWriter**
 - Char based streams
 - Both extends the abstract classes **Reader**, respectively **Writer**
 - Associated to objects of type **InputStream**, respectively **OutputStream**;
- **BufferedReader** and **BufferedWriter**
 - Associated with objects of type **Reader**, respectively **Writer**
 - Are using buffers in order to implement input/output operations

Files

Context class:

- FileInputStream **openFileInput**(String fileName)
- FileOutputStream **openFileOutput**(String fileName, int mode)
 - Context.MODE_PRIVATE/ Context.MODE_APPEND
- boolean **deleteFile**(String fileName)
- String[] **fileList**()
- File **getDir**(String dirName, int mode)

Internal Storage Example

```
private void writeInternalStorage() throws
IOException {

    String message = "These are sample contents
written to internal file";

    FileOutputStream outputStream =
openFileOutput("sample.txt",
Context.MODE_PRIVATE);
    outputStream.write(message.getBytes());
    outputStream.close();

    Toast.makeText(this, "Write Successful",
Toast.LENGTH_LONG).show();
}
```

```
private void readInternalStorage()
throws IOException {
    FileInputStream inputStream =
openFileInput("sample.txt");
    InputStreamReader streamReader
= new
InputStreamReader(inputStream);
    BufferedReader reader = new
BufferedReader(streamReader);
    String message =
reader.readLine();
    Toast.makeText(this, message,
Toast.LENGTH_LONG).show();
}
```

External files

- `android.permission.WRITE_EXTERNAL_STORAGE`
- Checking the availability of the external storage
- Checking the permission for accessing and working with the external storage

External Storage Example

```
private void writeExternalStorage() throws
IOException {
    String message = "These are the contents
which would be written to external file";

    File file = new
File(Environment.getExternalStorageDirectory(),
"sample.txt");

    if (!file.exists()) {
        file.createNewFile();
    }

    FileOutputStream outputStream = new
FileOutputStream(file);
    outputStream.write(message.getBytes());
    outputStream.close();

    Toast.makeText(this, "Write Successful",
Toast.LENGTH_LONG).show();
}
```

```
private void readExternalStorage() throws
IOException {

    File file = new
File(Environment.getExternalStorageDirectory(),
"sample.txt");
    FileInputStream inputStream = new
FileInputStream(file);

    InputStreamReader streamReader = new
InputStreamReader(inputStream);
    BufferedReader bufferedReader = new
BufferedReader(streamReader);

    String message = bufferedReader.readLine();

    Toast.makeText(this, message,
Toast.LENGTH_LONG).show();
}
```

Resources

- C. Boja, C. Ciurea, M. Doinea – Android Mobile Applications: A Practical Development Guide, ASE Press, 2015, 404 pp., ISBN 978-606-34-0033-9
- S. Komatineni, D. MacLean – Pro Android 4, Apress, 2012
- R. Meier – Professional Android 4 Application Development, Wiley, 2012
- P. Pocatilu, Programarea dispozitivelor mobile, Editura ASE, 2012
- P. Pocatilu, I. Ivan ș.a. – Programarea aplicațiilor Android, Editura, ASE, 2015
- <http://developer.android.com>